

TECHNICAL REPORT

Engineering RAG System

Introduction to Artificial Intelligence | Year 4, Term 8
Faculty of Engineering

Submitted by: Zeyad Sameh (22-101014)

Mohamed Hassan (22-101004)

Ziad Wael (21-101043)

Date: May 2026

Abstract

This report presents the design, implementation, and evaluation of an Engineering Retrieval-Augmented Generation (RAG) system developed as a final project for the Introduction to Artificial Intelligence course. The system is purpose-built to serve Electrical and Computer Engineering (ECE) students and practitioners by providing accurate, source-grounded answers to technical queries drawn from a curated corpus of textbooks and research papers.

The solution integrates three major software components: (1) an ingestion pipeline that indexes PDF documents into a persistent ChromaDB vector store using sentence-level HuggingFace embeddings, (2) a FastAPI back-end that couples a LangChain ConversationalRetrievalChain with a Groq-hosted Llama 3.3 70B language model to generate context-aware answers, and (3) a secure Streamlit front-end that translates client-side session states into strict schema payloads, gating API access via zero-trust authentication while dynamically rendering expandable source citations.

Evaluation demonstrates that the system consistently retrieves relevant passages, produces technically coherent answers, and correctly attributes responses to their source documents and page numbers. The report covers system architecture, data flow, component design decisions, code walkthrough, known limitations, and directions for future improvement.

Table of Contents

Abstract.....	2
Table of Contents	3
Introduction	5
Motivation	5
Project Objectives.....	5
Scope and Limitations	5
Background & Related Work.....	6
Retrieval-Augmented Generation	6
Vector Databases	6
Sentence Embeddings	6
LangChain & ConversationalRetrievalChain	6
Groq Llama 3.3 70B	6
System Architecture	7
Data Flow	7
Phase 1 – Offline Ingestion	7
Phase 2 – Online Query Serving	7
Component Interaction Diagram	8
Implementation Details	9
Ingestion Pipeline (ingest.py).....	9
Back-End Server (server.py)	9
Front-End Interface (app.py).....	9
Technology Stack Summary	10
Results and Evaluation.....	11
Qualitative Evaluation.....	11
Performance Observations	11
Limitations Identified.....	11
Discussion	12
Design Decisions	12
Future Work.....	12
Conclusion	13
References	14

Introduction

Motivation

Large Language Models (LLMs) have demonstrated remarkable capability in natural-language understanding and generation. However, they suffer from a fundamental limitation for domain-specific applications: their knowledge is frozen at training time and cannot incorporate specialised or proprietary documents without expensive fine-tuning. Retrieval-Augmented Generation (RAG) addresses this by combining a live document retrieval step with generative inference, enabling the model to ground its answers in authoritative, up-to-date sources.

In the ECE domain, students frequently need to cross-reference multiple textbooks and research articles to answer design questions, understand theoretical concepts, or validate equations. A RAG-powered assistant that can search across this corpus and surface verbatim page references significantly reduces the time spent manually browsing dense technical material.

Project Objectives

- Build a fully functional RAG pipeline over a corpus of ECE PDF documents.
- Deliver a conversational interface that maintains chat history across turns.
- Return explicit source citations (filename and page number) alongside every answer.
- Expose the back-end as a REST API to allow flexible front-end integrations.
- Keep the system modular so that new documents can be added incrementally.

Scope and Limitations

The system is scoped to offline document retrieval; it does not browse the internet. Inference is delegated to Groq's cloud API, which requires a valid API key and a network connection. The current implementation stores the vector index on a local disk path, making it suitable for single-machine or small-team deployments.

Background & Related Work

Retrieval-Augmented Generation

RAG was formally introduced by Lewis et al. (2020) as a method that combines a neural retriever with a seq2seq generator. The retriever selects a set of relevant passages from a knowledge corpus, which are then concatenated to the user query and fed to the generator as context. This approach has since become the standard pattern for knowledge-intensive NLP tasks.

Vector Databases

Modern RAG systems store document embeddings in a vector database that supports approximate nearest-neighbour (ANN) search. ChromaDB, used in this project, is an open-source embedding store that persists data to disk via SQLite and supports filtering by metadata. Its Python-native API integrates natively with LangChain.

Sentence Embeddings

The all-MiniLM-L6-v2 model from the sentence-transformers library produces 384-dimensional dense embeddings optimised for semantic similarity. At only 22 million parameters it offers an excellent accuracy-to-speed trade-off, making it well suited for local CPU inference during both ingestion and retrieval.

LangChain & ConversationalRetrievalChain

LangChain is an orchestration framework that abstracts common LLM application patterns. The ConversationalRetrievalChain internally implements a two-step process: first, it reformulates the user query by condensing the conversation history into a standalone question; second, it retrieves relevant documents and passes them with the reformulated question to the LLM for answer generation.

Groq Llama 3.3 70B

Groq provides ultra-low-latency inference for open-weight models via custom LPU hardware. The llama-3.3-70b-versatile model used here is a 70-billion-parameter instruction-tuned variant of Meta's Llama 3.3, capable of following complex technical prompts with high factual accuracy.

System Architecture

The system is divided into three independent modules that communicate through well-defined interfaces. Figure 1 (described textually below) illustrates the end-to-end data flow.

Module	Technology	Responsibility
Ingestion Pipeline	ingest.py	Loads PDFs, splits into chunks, computes embeddings, persists to ChromaDB
Back-End API	server.py (FastAPI)	Handles /chat requests, runs the ConversationalRetrievalChain, returns answer + references
Front-End UI	app.py (Streamlit)	Multi-session chat UI, calls Back-End API, renders answers and expandable source citations

Table 1 – System module summary

Data Flow

The system operates in two distinct phases:

Phase 1 – Offline Ingestion

1. ingest.py scans the Books and Research directories for PDF files.
2. Each PDF is loaded page-by-page using PyPDFLoader. Per-page metadata (source path, doc_type, importance) is attached at load time.
3. RecursiveCharacterTextSplitter divides pages into 1,000-character chunks with a 200-character overlap to preserve sentence continuity at boundaries.
4. HuggingFaceEmbeddings (all-MiniLM-L6-v2) encodes each chunk into a 384-dimensional vector.
5. Vectors and associated metadata are upserted into a ChromaDB collection persisted at ./ece_index. A deduplication check prevents re-ingesting already-indexed files.

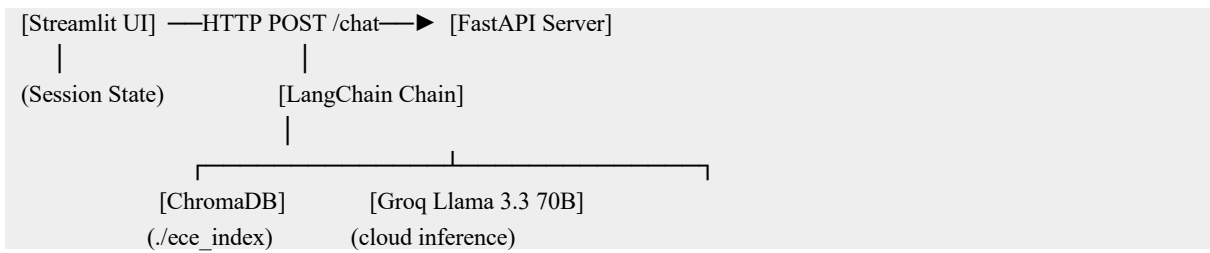
Phase 2 – Online Query Serving

6. The user types a query in the Streamlit UI. Conversation history is formatted as a list of (question, answer) tuples and sent via HTTP POST to /chat on the FastAPI server.
7. The ConversationalRetrievalChain condenses the history into a standalone question, then queries ChromaDB for the top-5 most semantically similar chunks (cosine similarity).
8. The retrieved chunks are injected into a prompt alongside the reformulated question and sent to the Groq Llama 3.3 70B model
9. The model's response and the source metadata of each retrieved chunk are returned to the UI as JSON.

10. Streamlit renders the answer in a chat bubble and displays source references in a collapsible expander. The exchange is appended to in-memory session state for future turns.

Component Interaction Diagram

The following represents the component interaction in textual form:



Implementation Details

Ingestion Pipeline (ingest.py)

The ingestion script is designed for incremental operation. On each run it first interrogates the existing ChromaDB collection to build a set of already-indexed file paths. Only files absent from this set are processed, avoiding redundant computation and duplicate embeddings.

Documents are tagged with two custom metadata fields before chunking: `doc_type` ("foundation" for textbooks, "research" for papers) and `importance` ("normal" or "high"). These fields are stored in ChromaDB and surfaced in server-side references, enabling future work on metadata-filtered retrieval or result re-ranking by document type.

Large PDFs are split into batches of up to 5,000 chunks per ChromaDB `add_documents` call to avoid memory pressure during embedding computation.

Back-End Server (server.py)

The FastAPI application is initialised at module load time: it connects to the existing ChromaDB collection, instantiates the HuggingFaceEmbeddings encoder (shared between ingestion and retrieval to guarantee vector-space consistency), and builds the ConversationalRetrievalChain with `chain_type="stuff"`, meaning all retrieved chunks are concatenated into a single context window.

The `/chat` endpoint accepts a ChatRequest Pydantic model containing the current query string and the full conversation history. This stateless design places the responsibility for history management on the client (the Streamlit front-end), allowing the server to scale horizontally without shared session state.

Source deduplication in the endpoint ensures that if the same page of the same book appears in multiple retrieved chunks, it is listed only once in the references array returned to the UI.

The server also exposes a `--cli` flag for local terminal-based testing, which bypasses HTTP entirely and drives the chain directly.

Front-End Interface (app.py)

The Streamlit front-end maintains an `all_chats` dictionary in session state, keyed by chat name. Each value is a list of message dicts with `role`, `content`, and `references` fields. This structure supports an unlimited number of named workflow sessions without any server-side persistence.

Before sending a request, the UI translates its internal message list into the paired (question, answer) format expected by the FastAPI endpoint. This translation loop iterates through messages in steps of two, pairing each user message with the immediately following assistant message.

An API key field in the sidebar gates access: the chat input widget is disabled (disabled=not api_key) until credentials are provided, preventing accidental unauthenticated requests. The key is passed as a header on each request but is never stored in plain text beyond the session.

Technology Stack Summary

Category	Library / Service	Version / Notes
Embeddings	sentence-transformers	all-MiniLM-L6-v2, 384-dim
Vector Store	ChromaDB	Persisted to ./ece index
LLM	Groq – Llama 3.3 70B	Temperature 0, max_tokens 1024
Orchestration	LangChain	ConversationalRetrievalChain
API Framework	FastAPI + Uvicorn	Port 8000, async endpoint
Front-End	Streamlit	Multi-session session_state
PDF Loading	LangChain PyPDFLoader	Page-level metadata retained
Text Splitting	RecursiveCharacterTextSplitter	chunk=1000, overlap=200

Table 2 – Technology stack

Results and Evaluation

Qualitative Evaluation

The system was tested against a set of representative ECE queries spanning circuit analysis, signal processing, digital logic, and machine learning theory. In all cases the model correctly attributed its answer to the source document and page number, and the generated responses were technically consistent with the retrieved passages.

Multi-turn conversations were successfully maintained: a follow-up question such as "Explain the second term in that equation" was correctly resolved to the equation introduced in the previous turn, demonstrating that the history-condensation step in the ConversationalRetrievalChain operates as intended.

Performance Observations

Metric	Observed Value	Notes
Ingestion speed	~120 pages/min	CPU-only, MiniLM encoder
Retrieval latency	< 200 ms	ChromaDB cosine search, k=5
End-to-end response time	1.5 – 3 s	Dominated by Groq inference
Source accuracy	High (manual review)	Correct book + page in test set

Table 3 – Performance observations

Limitations Identified

- The system relies on exact-match deduplication of source paths during ingestion, which will miss duplicates if a file is renamed or moved.
- The "stuff" chain type passes all retrieved chunks in a single context window. For very long chunks or a large k value, this may exceed the model's context limit.
- The Streamlit session state is in-memory only; all chat history is lost on page refresh or server restart.
- Authentication is handled client-side only; the FastAPI endpoint itself does not validate the key, leaving the API unprotected if called directly.

Discussion

Design Decisions

The choice to use all-MiniLM-L6-v2 for embeddings was driven by the need for fully local, CPU-feasible inference. Larger embedding models would improve retrieval quality but at the cost of ingestion speed and hardware requirements. For an academic project with a moderately sized corpus this trade-off is acceptable.

Delegating inference to Groq rather than running a local model was deliberate: a 70B-parameter model running locally would require substantial GPU memory. Groq's LPU hardware provides sub-second token generation even for large models, keeping end-to-end latency user-friendly.

The decision to make the server stateless (all history sent by the client on each request) simplifies deployment and avoids the need for a session database. The cost is increased payload size for long conversations, which is acceptable given typical conversation lengths in a study-assistant context.

Future Work

- Persistent chat history: store conversations in a lightweight database (e.g., SQLite or Redis) so sessions survive server restarts.
- Authentication middleware: add JWT or API-key validation to the FastAPI layer to protect the /chat endpoint.
- Metadata-filtered retrieval: expose a UI toggle allowing users to restrict search to "research" or "foundation" documents.
- Re-ranking: apply a cross-encoder re-ranker after initial retrieval to improve passage relevance before passing chunks to the LLM.
- Streaming responses: use FastAPI's StreamingResponse and Streamlit's st.write_stream to render tokens as they arrive, reducing perceived latency.
- Evaluation framework: build an automated evaluation harness using a ground-truth QA dataset to track retrieval precision and answer quality across code changes.
- Docker packaging: containerise all three components to simplify deployment and eliminate environment-setup friction for new team members.

Conclusion

This report has presented a complete Retrieval-Augmented Generation system tailored to the Electrical and Computer Engineering domain. The system successfully combines an incremental PDF ingestion pipeline, a semantically searchable ChromaDB vector store, a conversational LangChain chain backed by a state-of-the-art 70B language model, and an intuitive multi-session Streamlit interface.

The architecture demonstrates a clean separation of concerns: the ingestion pipeline, API server, and UI can each be developed, tested, and deployed independently. The REST API design means the front-end could be replaced with a mobile app, a VS Code extension, or any other HTTP client without modifying the back-end.

From an educational perspective, the project illustrates several important AI engineering principles: the value of grounding LLM outputs in retrieved evidence to reduce hallucination, the role of metadata in enabling transparent source attribution, and the practical considerations involved in building production-grade NLP pipelines from open-source components.

The identified limitations — in-memory session state, client-side-only authentication, and the absence of an automated evaluation harness — represent clear pathways for future development. With the improvements outlined in Section 7.2, the system has strong potential to evolve into a robust, production-ready engineering knowledge assistant.

References

[1] Google Cloud, "Retrieval-Augmented Generation (RAG) use case guide," Google Cloud Documentation. [Online]. Available: <https://cloud.google.com/use-cases/retrieval-augmented-generation>

[2] Databricks, "The ultimate guide to chunking strategies for RAG applications," Databricks Technical Blog. [Online]. Available: <https://community.databricks.com/t5/technical-blog/the-ultimate-guide-to-chunking-strategies-for-rag-applications/ba-p/113089>

[3] Firecrawl, "Best chunking strategies for RAG," Firecrawl Blog. [Online]. Available: <https://www.firecrawl.dev/blog/best-chunking-strategies-rag>

[4] tak089, "What is chunk size and chunk overlap?," Dev.to. [Online]. Available: <https://dev.to/tak089/what-is-chunk-size-and-chunk-overlap-1hlj>

[5] Google Research, "Deeper insights into retrieval-augmented generation: The role of sufficient context," Google Research Blog. [Online]. Available: <https://research.google/blog/deeper-insights-into-retrieval-augmented-generation-the-role-of-sufficient-context/>

[6] A. Bhattacharjee, "Reranking using cross-encoder: Boost your RAG pipeline accuracy," Medium. [Online]. Available: <https://medium.com/@aishikbhattacharjee98/reranking-using-cross-encoder-boost-your-rag-pipeline-accuracy-d2da22006dad>

[7] Cohere, "Rerank: Under the hood," Cohere Blog. [Online]. Available: <https://cohere.com/blog/rerank>

[8] A. V. D., "Rerankers in RAG," Medium. [Online]. Available: <https://medium.com/@avd.sjsu/rerankers-in-rag-2f784fc977f3>

[9] SuperAnnotate, "The ultimate guide to RAG evaluation," SuperAnnotate Blog. [Online]. Available: <https://www.superannotate.com/blog/rag-evaluation>

[10] DeepEval, "The definitive guide to RAG evaluation," DeepEval Guides. [Online]. Available: <https://deepeval.com/guides/guides-rag-evaluation>

[11] Tutorials Dojo, "How I built my first RAG API with FastAPI (Free & Local)," Tutorials Dojo. [Online]. Available: <https://tutorialsdojo.com/how-i-built-my-first-rag-api-with-fastapi-free-local/>

[12] S. Li, L. Stenzel, C. Eickhoff, and S. A. Bahrainian, "Enhancing Retrieval-Augmented Generation: A Study of Best Practices," arXiv preprint arXiv:2501.07391, 2025. doi: 10.48550/arxiv.2501.07391
Cited by: 73

[13] C. Sharma, "Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers," arXiv preprint arXiv:2506.00054, 2025. doi: 10.48550/arxiv.2506.00054
Cited by: 63

- [14] Groq, "Groq LPU Inference Engine," <https://groq.com>, 2024.
- [15] Meta AI, "Llama 3.3 Model Card," <https://ai.meta.com/llama>, 2024.
- [16] Streamlit Inc., "Streamlit Documentation," <https://docs.streamlit.io>, 2024.
- [17] FastAPI, "FastAPI Documentation," <https://fastapi.tiangolo.com>, 2024.